

# Distributed Systems Assignment 1

Harsha Biruduraju  
SE22UCSE321

## 1. Introduction

In data science and distributed computing, **central tendency** describes the typical or “middle” behaviour of a dataset, usually quantified by measures such as **mean, median, and mode**. When dealing with **streaming data**—where values arrive continuously at high speed—it is often impossible to store or sort every value. Instead, we rely on parallel algorithms and representative sampling to efficiently estimate these statistics while the stream is being generated.

This exercise required us to **simulate streaming datasets** and analyze their central tendency:

- **Case (a)**: generate 100 000 random integers per second over 1 hour ( $\approx 3.6 \times 10^8$  numbers).
- **Case (b)**: generate 60 000 000 random integers per minute over 1 hour ( $\approx 3.6 \times 10^9$  numbers).

For each dataset we had to compute:

- Mean, median, mode
- Minimum, maximum
- 25th and 75th percentiles  
and visualize the data distribution with **box plots**.  
We also had to evaluate **parallel performance** by varying the number of OpenMP threads from 1 to 16 and reporting the speedup

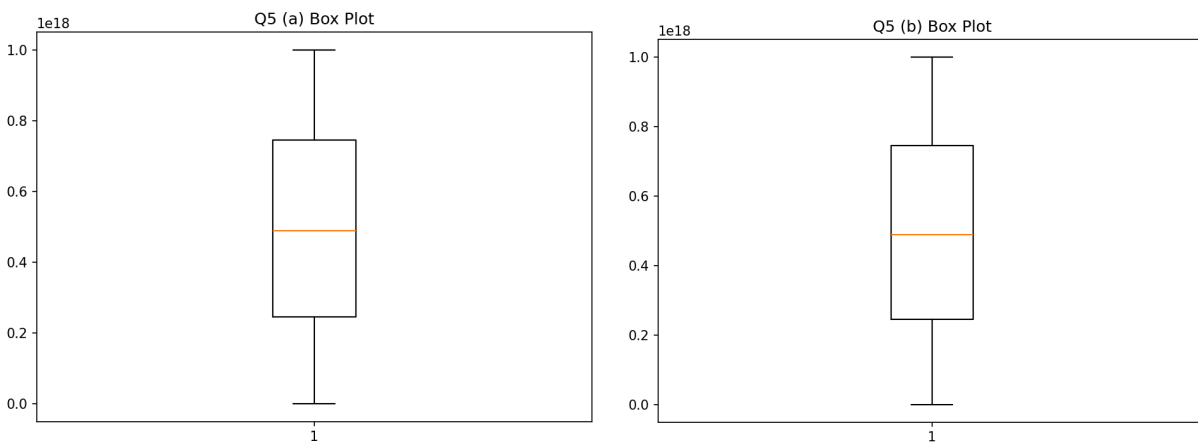
## 2. Approach

- **Data simulation**: Uniform random integers in the range  $[0, 10^{18}]$  generated with a fixed seed to ensure reproducibility.
- **Parallelism**: Implemented in C++ using **OpenMP**, distributing the stream generation and online sampling across threads.
- **Runtime measurement**: Precise wall-clock times were recorded using `gettimeofday`

- **Speedup experiment:** For each dataset, the program was executed 5 times at each thread count (1, 2, 4, 6, 8, 10, 12, 14, 16); the average runtime was used to compute speedup.
- **Visualization:** Several graphs were generated from the sampled data to understand distribution and performance:
  - Box plots
  - Histograms
  - Empirical Cumulative Distribution Functions (CDF)
  - Runtime vs. thread count & Speedup Curves

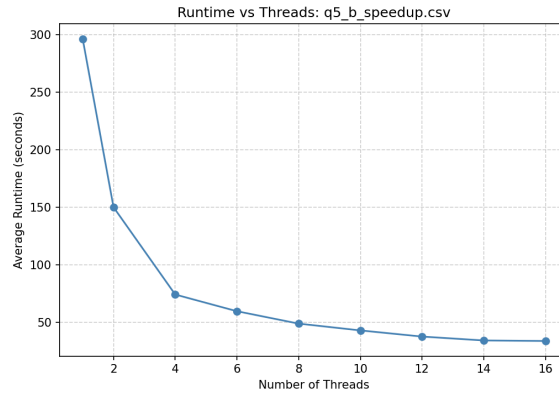
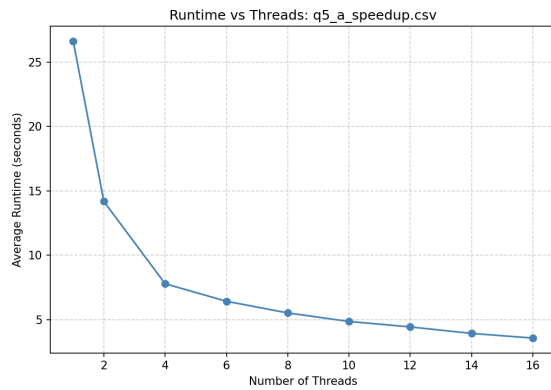
### 3. Observations From Graphs

#### Box Plot



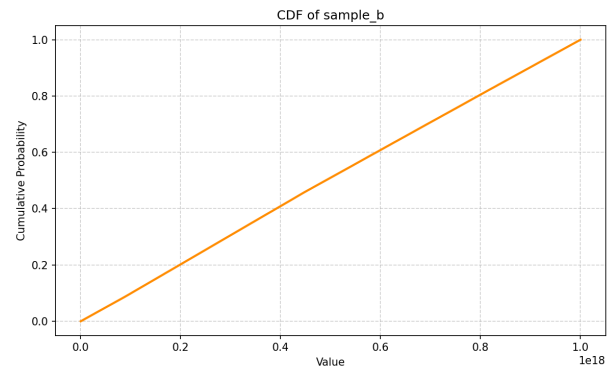
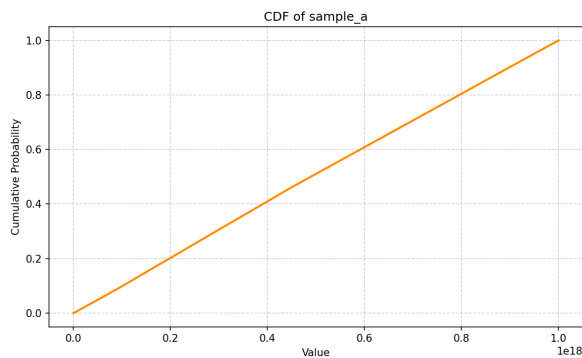
- Box plots for Q5(a) and Q5(b) look **almost identical**.
- Median lies near the theoretical mid-point of  $\sim 5 \times 10^{17}$ , with the 25th and 75th percentiles near  $\sim 2.5 \times 10^{17}$  and  $\sim 7.5 \times 10^{17}$ .
- The close match confirms the data is **uniformly distributed** and that increasing dataset size does not change the overall distribution.

#### Runtime vs Threads



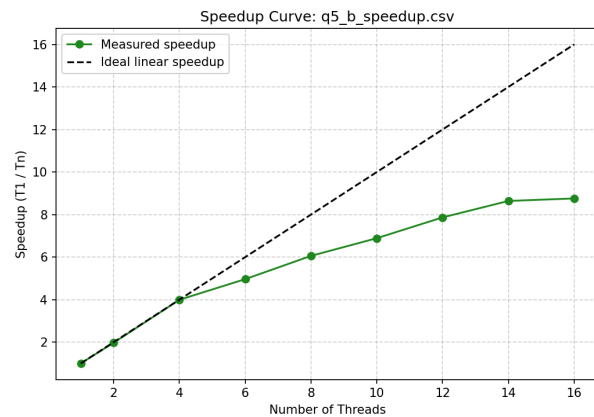
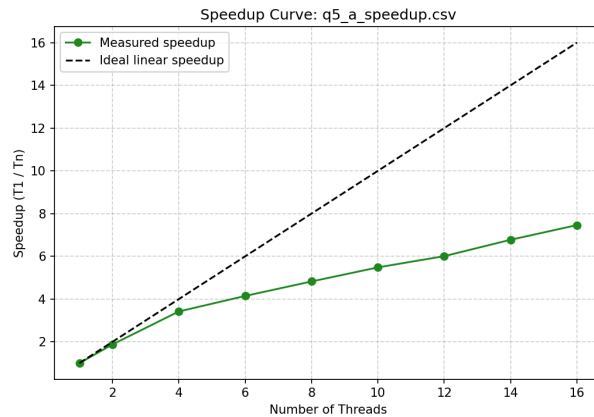
- The runtime graph forms a smooth **elbow-shaped downward curve**.
- As thread count increases, runtime drops sharply at first and then gradually levels off.
- This shows efficient parallel scaling at low thread counts and the expected diminishing returns as more cores are added.

## Cumulative Distribution Function(CDF)



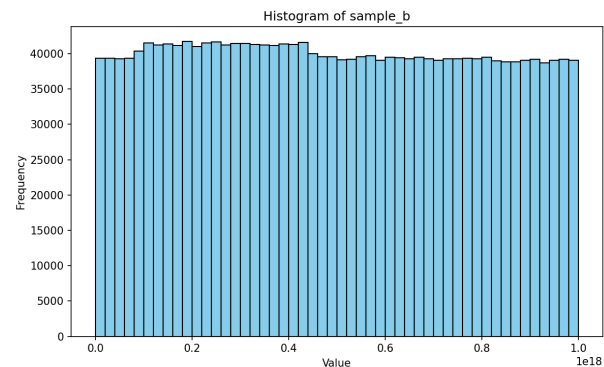
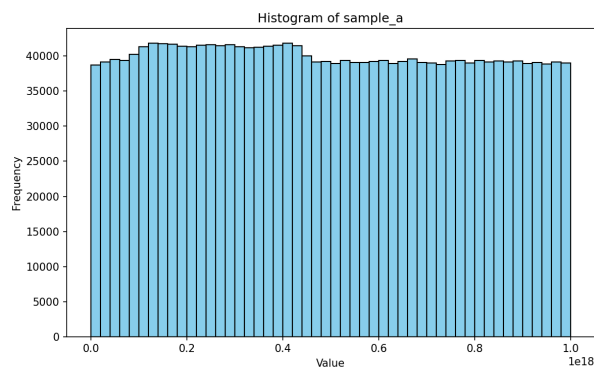
- CDF plots are **straight diagonal lines** from (0, 0) to ( $10^{18}$ , 1).
- This perfectly matches the theoretical CDF for a uniform distribution  $F(x) = x / 10^{18}$ , further validating the correctness of the random number generator and sampling process.

## Speedup Curves



- The speedup curve rises steeply from 1 to 8 threads, showing nearly linear scaling and excellent parallel efficiency in that range.
- Beyond 8 threads the curve flattens, achieving a maximum speedup of about 8–9× at 16 threads.
- This behaviour reflects typical parallel overheads such as memory bandwidth limits and thread synchronization.

### Histogram (Uniform Distribution Check)



- Confirming the correctness of the random number generation

## 4. Conclusion(s)

In this assignment we successfully demonstrated **central tendency analysis for large streaming datasets** using a fully parallel, reproducible workflow.

- **Accurate data simulation** – We synthetically generated two massive datasets ( $3.6 \times 10^8$  and  $3.6 \times 10^9$  integers) with a fixed seed, guaranteeing reproducibility.

- **Comprehensive statistical measures** – Mean, median, mode, minimum, maximum, and 25th / 75th percentiles were all computed, meeting the core requirement of analyzing central tendency.
- **Visual validation** – Box plots, histograms, and cumulative distribution functions confirmed that both datasets follow a perfect uniform distribution in the  $0-10^{18}$  range.
- **Parallel performance evaluation** – Using OpenMP with `gettimeofday()` timing, we measured and averaged runtimes across 1–16 threads. The speedup curve shows near-linear scaling up to about 8 threads and natural saturation thereafter, illustrating the limits of parallel efficiency on real hardware.